

# cute 之 Hopper TMA

在[Hopper架构](#)之前所有的数据拷贝都是多个线程共同完成的，它包含每个线程完成大致相似的地址计算，每一个线程最多单次完成16字节的数据加载，通过多次循环完成更大规格的数据搬运，但是在Hopper及其之后的架构中，为了提高数据的加载效率，减少复杂且容易出错的加载流程，NVidia在Hopper架构中将数据搬运任务抽象成独立的硬件单元TMA（Tensor Memory Accelerator）。借助TMA的能力，Hopper上可以实现高效的数据搬运。本文首先介绍了传统的[SIMT编程模型](#)以及该模型下的数据搬运，然后重点介绍了Hopper上的TMA能力和编程模型的变更，详细介绍了它的TMA描述符、发起拷贝任务等特点，最后文章介绍了cute对TMA能力的封装和抽象。

## SIMT编程模型中的数据搬运

CUDA从一开始就选定了SIMT编程模型，从编程逻辑角度由宏观到微观看，每一个程序（kernel）由一个线程网格（kernel）组成，每一个线程网格由一个或多个线程块（block，也称为thread block）组成，每一个线程块由一个或多个线程（thread）组成（如图1）。线程网格具有一定的局部性，在线程网格内的多线程之间可协同的完成计算和同步。

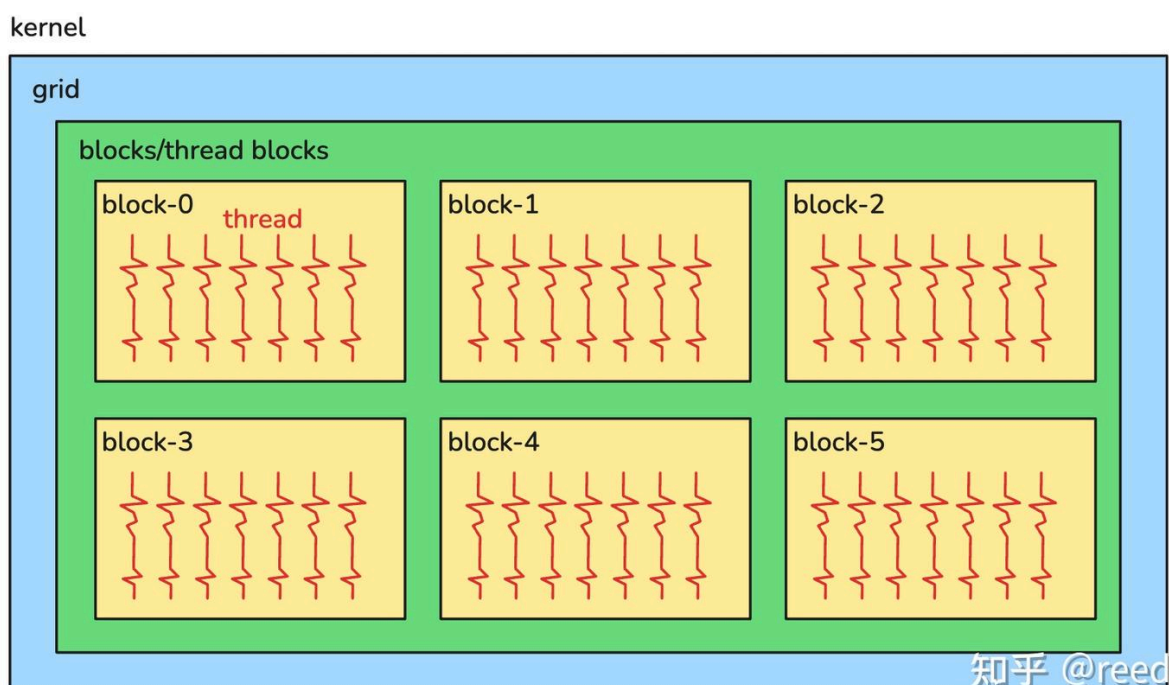


Figure 1. SIMT Programming Model

编程模型是软件模型，其本质是对硬件结构和能力的抽象。SIMT编程模型之所以这样抽象，是由NVidia硬件的设计模式决定的。如图2所示，NVidia GPU硬件架构整体分为四层，最内层的执行单元

为带有Predication能力的SIMD核心，其可以在单个cycle内向量化的完成浮点运算，图中FP32核心可以在一个周期内完成32个32bit的符合IEEE标准的浮点乘法、加法、乘加运算，可以通过Predicate来决定该lane是否执行。和FP32功能单元类似，NVidia硬件还提供了整数计算单元（INT32）、64bit浮点计算单元（FP64）、矩阵计算单元（TensorCore）、数据加载单元（LSU）和特殊函数单元（SFU，可以完成正弦、余弦、倒数、开平方、E指数、对数等计算），这些不同计算类型的单元组合的在一起形成一个多功能计算能力，同时组合上Warp调度器（warp scheduler）和分派单元（dispatch unit）共同形成subcore，其可以实现warp级别调度和执行能力。4个subcore组合在一块形成更大的计算能力，同时附上L1数据缓存（L1 data cache）和共享内存（shared memory）形成流处理器（Stream Multiprocessor，简称SM），在SM上会有Barrier硬件单元能够协同多个subcore的数据同步和执行进度，共享内存提供本地局部的数据复用。多个SM组合在一起同时附上L2缓存（L2 Cache）和高带宽内存（High Bandwidth Memory）形成GPU设备（device）。

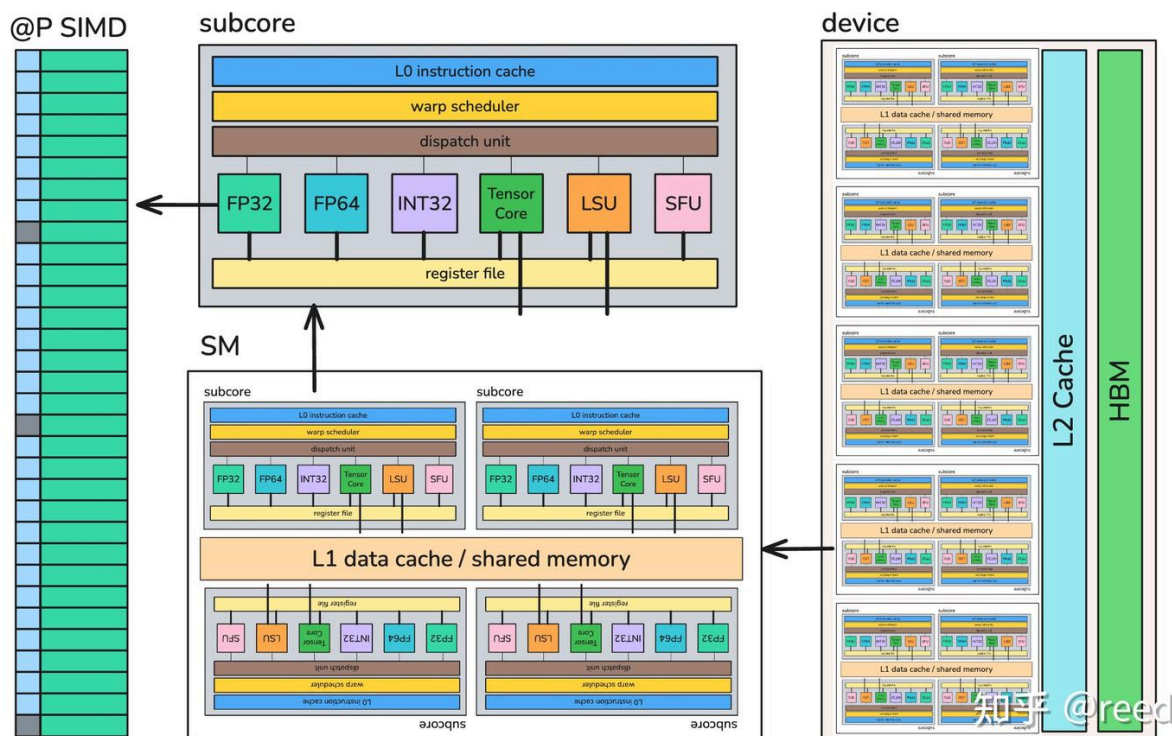


Figure 2. NVidia GPU Hardware Architecture Abstraction

软件上的Grid是硬件Device的抽象，并且通过调度单元使得数量有限的SM可以支撑远超其规格的grid大小。软件上的block是硬件SM的抽象，block内包含多个warp，block内线程间可以通过共享内存共享数据，可以做线程同步，本质则是硬件在SM层级提供了可编程的cache机构shared memory，同时提供了执行同步的barrier能力。当一个block使用的资源不多的时候，一个SM可以承载多个并行的block来提高资源利用效率。软件上的warp是对硬件subcore的抽象，单个warp内可以进行并行的计算，可以完成多种类型指令的计算任务。而带Predicate的SIMD核心则是线程的抽象，通过Predicate来mask掉某些不需要执行的lane，从而实现SIMT语义。

当我们使用GPU完成一个特定问题的求解时，本质是将任务拆分为可并行的更小的子任务，这些任务最终被分配到一个并行执行的FP32或者INT32等核心上执行。具体到每一个线程，其根据其所在的block号和thread号（CUDA中的blockIdx和threadIdx）来映射一个问题域的很小的计算逻辑。针对数据计算和加载都是这样的。整体可以表示为  $T = P(\text{threadIdx}, \text{blockIdx})$ 。根据任务的划分使用FP32/INT32/LSU等来完成对数据的加载和计算。

以上便是SIMT模型下GPU到工作逻辑，针对于深度学习场景，其计算面对的输入输出通常是高维的Tensor，而在任务划分后所需要完成的数据拷贝也是高维空间中的一块数据，如常用的GEMM类计算，在Ampere架构上每一个线程会负责的任务有：

1. 将数据加载到共享内存（使用INT32）
2. 读取共享内存数据读取到寄存器
3. 使用TensorCore完成矩阵乘加计算
4. 将结果写出到共享内存
5. 读取共享内存数据到寄存器
6. 将寄存器数据写出到全局内存

不难发现以上GEMM计算相关的内存操作中涉及大量的共享内存的操作，原因是使用共享内存可以提高数据复用，减少对全局内存的访问量，减少对低带宽通路的占用，继而提升计算效率。但是针对全局内存和共享内存之间的读取和写入而言，其会涉及到大量的地址计算和繁琐的处理，以读取全局内存写入共享内存为例，其需要使用INT32单元计算全局内存上作为输入的数据的地址，使用INT32单元计算计算共享内存输出的地址，同时由于共享内存的bank结构，需要引入避免bank冲突的逻辑，也会消耗整数和逻辑ALU指令，除了做bank冲突处理，矩阵计算一般是按照块进行的，对于边界不能整除的场景（tile quantization），经常需要对读取进行mask掉，对于输出的共享内存部分则需要填充上零，也会引入额外的指令消耗。值得注意的是，在SIMT编程模型下，这些操作需要所有的线程参与。并且处理这类问题很容易出错，有统计表明，CUDA开发者90%的时间是在处理数据访问相关的问题，换句话说CUDA类程序员大部分时间是在处理illegal memory相关的问题。

## 高效的异步Tensor内存拷贝加速单元

针对SIMT意义下这种硬件计算浪费、编程错误率高、开发效率低的问题，NVidia在Hopper架构上给出了全局内存和共享内存之间数据拷贝的软硬件协同的高效的解决方案。硬件方面提供独立的张量内存加速单元（Tensor Memory Accelerator简称TMA），它将数据拷贝相关功能进行抽象并高度集成，形成独立的硬件单元，使其能够完成从一维到五维的全局内存和共享内存之间的大块Tensor语义的拷贝，拷贝的同时能够配置避免共享内存bank冲突的Swizzle模式+，同时可以配置超过边界（Out Of Bound, OOB）时的填充模式，为了提高拷贝和计算可并行的效率，这些拷贝是异步执行的，即SM在发起拷贝任务后可以立即进行其他的任务，线程并不会因为拷贝而被阻塞。软件方面也

打破了之前SIMT的编程范式，不再需要所有的线程参与地址的计算和数据读取，而是只需要一个线程即可以发起大块的Tensor的拷贝任务。

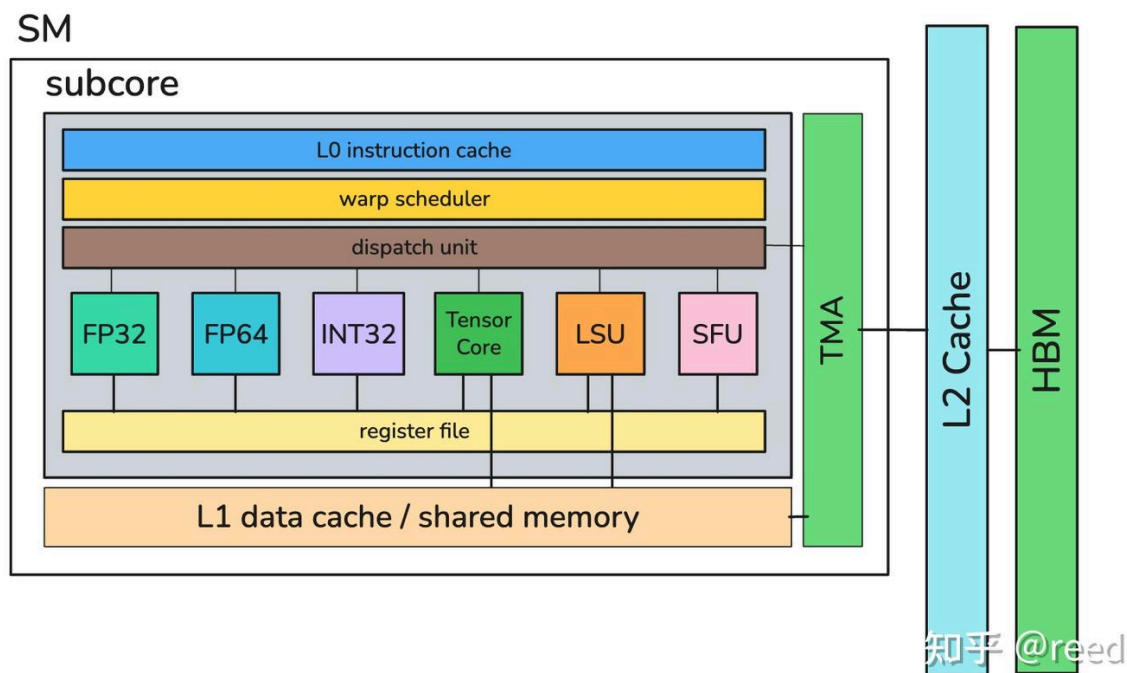


Figure 3. TMA Connection Illusttation

如图3所示，TMA在硬件结构上和SM一一对应，其被布置在SM内部，Dispatch单元可以向TMA单元发送数据拷贝指令，该指令发送后TMA则可以异步的进行数据拷贝，完成全局内存到SM内Shared Memory上的数据搬运。

## TMA描述符（TMA Descriptor）

使用TMA完成全局内存到共享内存到数据搬运需要两个阶段，第一阶段为定义TMA描述符，第二阶段为实施Copy阶段（Copy Operation）。TMA描述符（TMA Descriptor）是一段内存结构其描述了完成拷贝所需要的必要信息，如图4所示，TMA描述符中包含全局内存的描述信息，输入Tensor的全局内存地址，Tensor的Shape和Stride表示；以及单次拷贝的拷贝块的大小表示（如图CopyBox），和共享内存上Tensor的swizzle信息和oob信息。通过描述符中的信息TMA则可以获得输入Tensor的完整精准的描述以及实施拷贝时的需要拷贝的大小和数据量，同时知晓在向共享内存写入时应该如何做Swizzle。在拷贝实施阶段，即SM发射了拷贝任务，则需要SM向TMA提供TMA描述符位置信息，TMA单元感觉该位置读取上诉信息，还需要提供要拷贝的块的起点位置（如图xy所示）；当以共享内存作为目标时，还需要提供共享内存的地址和作为拷贝完成机制的**mbarrier+**的地址。

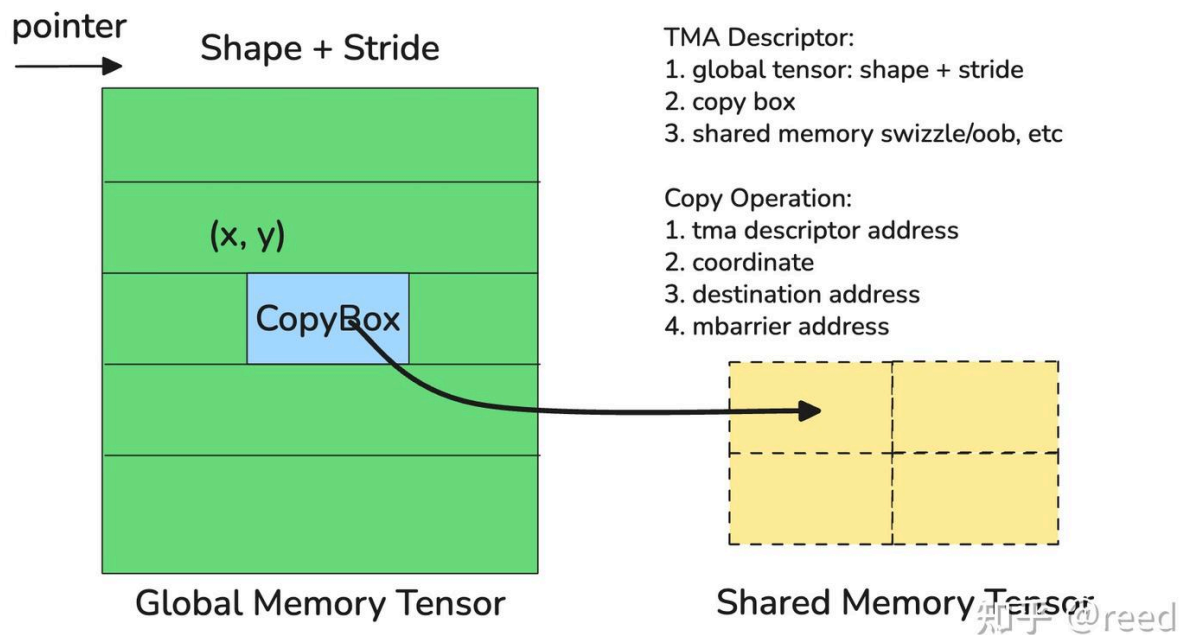


Figure 4. TMA operation mechanism

对于TMA描述符的构建，CUDA提供了特定的Driver API `cuTensorMapEncodeTiled`如下所示，其通过输入内存描述等信息，构建了TMA等描述符（tensorMap），在使用TMA完成拷贝时需要将该描述符传递给CUDA Kernel。

```
CUresult cuTensorMapEncodeTiled (
    CUtensorMap* tensorMap,
    CUtensorMapDataType tensorDataType,
    cuuint32_t tensorRank,
    void* globalAddress,
    const cuuint64_t* globalDim,
    const cuuint64_t* globalStrides,
    const cuuint32_t* boxDim,
    const cuuint32_t* elementStrides,
    CUtensorMapInterleave interleave,
    CUtensorMapSwizzle swizzle,
    CUtensorMapL2promotion l2Promotion,
    CUtensorMapFloat00Bfill oobFill);
```

目前TMA Descriptor传递给Kernel有三种形式。第一种使用`const __grid_constant__` 修饰符通过 Kernel Parameter传值的方法传递给kernel, 如下

```
__global__ void tma_desc_demo1(const __grid_constant__ CUtensorMap map);
```

第二种使用CUDA运行时API `cudaMemcpyToSymbol` 将描述符拷贝到 `__constant__` 修饰的device 变量上。

```
// device
__constant__ CUtensorMap device_tma_desc;

// Host
CUtensorMap host_tma_desc;
cudaMemcpyToSymbol(device_tma_desc, &host_tma_desc, sizeof(host_tma_desc));
```

第三种则是将TMA Descriptor存储在全局内存上, CUDA Kernel内部直接向TMA提供该描述符地址即可

```
__global__ void update_tma(CUtensorMap *desc) {
    // update TMA Descriptor fields
    // update global memory shape / stride
    // update copy box etc.
}

__global__ void tma_desc_demo3(CUtensorMap *desc) {
    // pass desc to TMA operation
}
```

以上三种形式适用于不同的场景, 第一种适用于从host端可以确定全局内存Tensor的情形, 如固定长度的linear层对应的gemm, 第二种和第一种类似但是需要多出一个拷贝类的API调用; 第三种为需要动态修改全局内存中Tensor的维度等信息时采用的模式, 如大模型中MoE (Mixture of Experts) 层中的GroupGEMM实现时, 矩阵的m轴是动态变化的, 这时一般采用第三种形式。不论是那种形式, 其本质都是一个大小为128字节的描述信息, 只是这个信息是相对固定的还是在运行时会动态更新和变化。就效率方面而言, 前两种方法CUDA驱动层能够确保描述符是在更高效的缓存系统中, 所

以TMA获取描述符时效率更高一下，第三种形式需要TMA单元从HBM经由L2 Cache来获取该描述符，会有一定的overhead，此时可以通过软件编程的角度在恰当的位置对描述符进行预取，使用的PTX指令和对应的SASS如下

```
// PTX
prefetch{.tensormap_space}.tensormap [a];

.tensormap_space =          { .const, .param };

// SASS
UTMACCTL.PF [UR4] ; // Uniform TMA Cache ConTraL PreFetch
```

上面的SASS指令也揭示了TMA单元为了提升操作效率会有TMA Descriptor的Cache机构。TMA在进行拷贝相关的任务时会有限从本地的cache机构中查找描述符信息，如果命中，则直接发起相应的拷贝任务，如果不能命中则会从更低级的存储机构中读取。

## 发起拷贝任务

TMA描述符精准的描述了全局内存的情况以及具体拷贝时的基本块，在具体的需要拷贝时可以通过PTX提供的指令实现，以全局内存到共享内存为例，具体指令为

```
cp.async.bulk.tensor.dim.dst.src{.load_mode}.completion_mechanism{.level::cache_hint}
                                     [dstMem], [tensorMap, tensorCoords], [mbar]
{, cache-policy}

.dst =          { .shared::cta }
.src =          { .global }
.dim =          { .1d, .2d, .3d, .4d, .5d }
.completion_mechanism = { .mbARRIER::complete_tx::bytes }
.load_mode =     { .tile, .im2col }
.level::cache_hint = { .L2::cache_hint }
```

其中cp来自copy的缩写，表示其为数据搬运指令，async表示该指令为异步指令，指令执行后并不代表copy完成，而只是代表copy任务被发起，bulk表示为大块内存拷贝区别于Ampere架构中的

cp.async指令（Ampere中的cp.async指令依然是SIMT范式的指令），tensor用以指示是面向tensor的copy，即由Tensor到Tensor到块拷贝，dim用以指示tensor的维度信息（亦即rank信息）只支持一维到五维，dst和src用以指定copy的目的和源内存类型，分别是thread block中共享内存和全局内存，load\_mode用以指示TMA的加载模式，除了本文重点介绍的tile模式，还有针对卷积运算（convolution）的im2col模式，tile模式即块到块的模式，输入输出等大小，completion\_mechanism表示完成机制，此处为mbarrier::complete\_tx::bytes，表示完成机制为mbarrier，当拷贝完成时TMA负责将拷贝的字节数写入到mbarrier中的transaction bytes字段中（参考cute之Hopper MBarrier），同时通知其完成，level::cache\_hint表示使用的L2的cache的驱逐策略，如evict\_norm、evict\_first、evict\_last，根据数据复用的时间、空间局部性，可以选择恰当的L2驱逐策略，以提升L2的利用效率。

以上介绍了TMA发起拷贝的modifier属性，具体操作数方面，有dstMem用于指定输出的地址，示例这里指共享内存的地址，tensorMap和tensorCoords用以指定TMA Descriptor到地址和拷贝块的坐标，这里会根据modifier中dim选择的不同而输出的参数数目不同，mbar用以指定完成时的通知机制mbarrier的地址，cache-policy为可选项，其用以指定L2的驱逐策略。

特别的针对一维的场景，可以不指定复杂的TMA Descriptor，而是指定全局内存的地址和size来完成大块数据的搬运，针对全局内存到共享内存拷贝，PTX提供的异步拷贝指令如下

```
// global -> shared::cta
cp.async.bulk.dst.src.completion_mechanism{.level::cache_hint}
                                [dstMem], [srcMem], size, [mbar] {, cache-policy}

.dst =                          { .shared::cta }
.src =                          { .global }
.completion_mechanism = { .mbarrier::complete_tx::bytes }
.level::cache_hint =          { .L2::cache_hint }
```

在modifier方面整体和tensor版本类似，只是不在需要tensor和dim modifier，参数方面也只需要指定global memory的地址和size即可，可以避免相对复杂的TMA Descriptor构建。虽然一维的拷贝不需要TMA Descriptor但是其整体工作范式依然是TMA模式而非传统的SIMT模式，其只需要一条指令即可以完成大块数据的拷贝，而无需所有线程参与拷贝任务。

## 完成机制

从以上的指令介绍可以看到，针对全局内存到共享内存采用了mbarrier作为完成机制，而针对共享内存到全局内存方向的拷贝则采用bulk group机制，即commit/wait机制，其类似Ampere中的cp.async，能够将之前发起的异步拷贝整体提交，提交之后通过wait指令可以实现对这些提交的完成确认，不同源和目的内存的完成机制如下图（引用自NVidia PTX文档）。

| .completion-mechanism | .dst             | .src         | Completion mechanism   |
|-----------------------|------------------|--------------|------------------------|
| .mbarrier::...        | .shared::cta     | .global      | mbarrier based         |
|                       | .shared::cluster | .global      |                        |
|                       | .shared::cluster | .shared::cta |                        |
| .bulk_group           | .global          | .shared::cta | Bulk async-group based |

Figure 5. TMA completion mechanism(From NVidia PTX ISA-9.1, chapter-9.7.9.25.4.1)

Swizzle机制

TMA Descriptor创建时为了避免共享内存的读取和或写入的冲突可以指定Swizzle模式，常用的有以下四种

```
CU_TENSOR_MAP_SWIZZLE_NONE,  
CU_TENSOR_MAP_SWIZZLE_32B,  
CU_TENSOR_MAP_SWIZZLE_64B,  
CU_TENSOR_MAP_SWIZZLE_128B
```

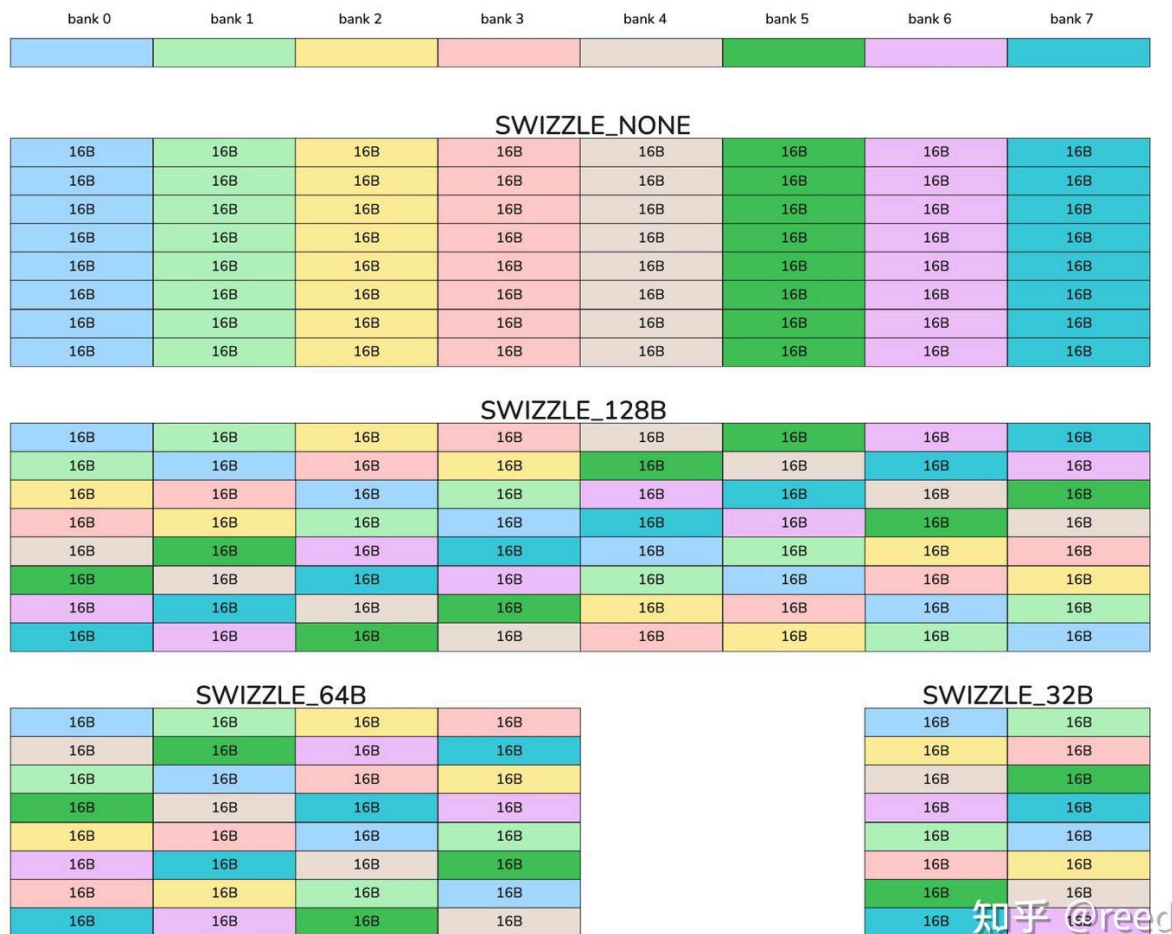


Figure 6. Swizzle Pattern for TMA

图6展示了shared memory的bank情况，此处最小的单元表示为16字节，可以认为shared memory有8个bank，不同的bank对应不同的颜色表示。对于不做Swizzle的情况，即SWIZZLE\_NONE, 横向128字节为不同的bank，纵向不做swizzle故而每行的bank颜色和第一行一致。针对SWIZZLE\_128B场景，即SWIZZLE的边界为128B，第二行的128B需要按照16B的为列号和行号进行异或得到swizzle后的bank号，即 $ibank = irow \oplus icol$ 。SWIZZLE\_64B的模式时，可以认为是SWIZZLE\_128B的第一行分为前后两个部分，第二部分，即后64B折返到第二行形成两行，第三行和第四行则可以认为是SWIZZLE\_128B的第二行的前后一半排列成两行。SWIZZLE\_32B以此类推。整体计算逻辑如下

```
int banks[sizeh][sizew];
for (int irow = 0; irow < sizeh; ++irow) {
    for (int icol = 0; icol < sizew; ++icol) {
        int ioffset = irow * sizew + icol;
        int irow1 = ioffset / 8;
```

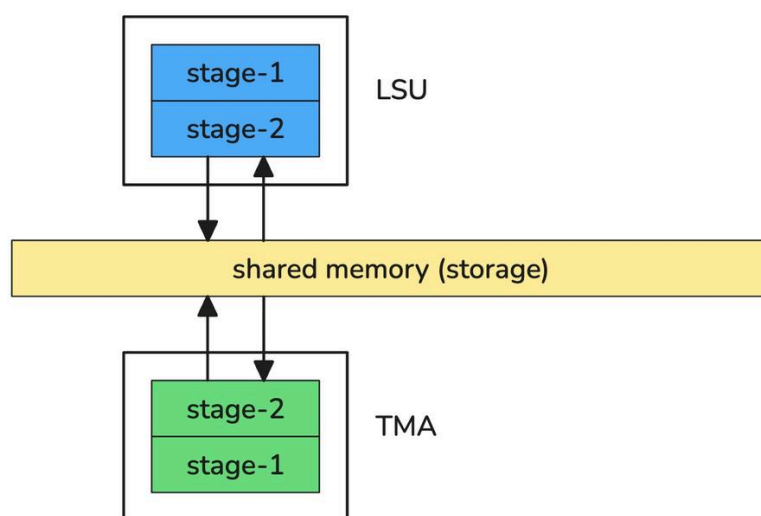
```

    int icol1 = ioffset % 8;
    banks[irow][icol] = irow1 ^ icol1;
}
}

```

## Async单元和可见性保障

通过上面的介绍我们了解到TMA可以异步的读写共享内存，传统的LSU也可以读写共享内存（即通过对\_\_shared\_\_ 内存进行读取和写入），但是TMA和LSU不共享操作流水线，这就造成TMA和LSU对shared memory操作的可见性方面需要额外的机制才能保障。



知乎 @reed

Figure 7. LSU and TMA has its own pipeline

TMA读取global的数据数据写入到shared memory中，此时通过mbarrier的wait机制可以可以确保线程块看到TMA读取的结果，通过LSU读取共享内存可以得到正确的结果。如果是通过LSU向共享内存写入数据，但是通过TMA读取，如果不进行额外操作则TMA可能读到错误的结果。如下示例，通过LSU先向共享内存写入数据然后调用线程块同步函数\_\_syncthreads，同步之后调用单线程的TMA读取共享内存，在时序上看似先向共享内存写入的数据，然后发起了TMA读取共享内存，但是在TMA发起后真正读取数据的时候，LSU对共享内存的写入可以依然在流水线上，并没有真正完成，这也不能保证对TMA对可见，所以TMA有可能读到错误的结果，如果在syncthreads后是一个普通的共享内存的读，则其由于其走相同的流水线，能够确保读取到正确的数据。

```

__shared__ float sdata[1024];

```

```
sdata[idx] = idx;
__syncthreads();

if(elected) {
    TMA::copy(sdata, ...);
}
```

针对以上场景，则需要显式的通过fence指令来确保内存事务的一致性，更新后的代码为

```
sdata[idx] = idx;
asm volatile("fence.proxy.async.shared::cta;\n");
__syncthreads();

if(elected) {
    TMA::copy(sdata, ...);
}
```

其中fence指令行能够确保fence之前的内存事务被追踪，确保TMA能够看到该内存屏障事务，\_\_syncthreads能够确保线程在执行层的同步，而不是可见性的同步，这样TMA发起的拷贝任务会等待fence事务完成后才能进行，能够得到正确结果，一定程度上可以认为有了fence语句之后，TMA可以看到LSU的写入完成并且会等待写入完成后才能开始其对共享内存的读取，所以能够得到正确的结果。

## 其他方面

除了上面介绍的TMA的基本的概念和用法，TMA还能提供Cluster内的Multicast能力，即读取一份数据可以组播给cluster内的多个block；除了上面提到的tile的拷贝能力，TMA还提供了针对convolution计算模式的image2column能力，完成复杂的坐标变换和数据排列。同时在TMA读取共享内存写出全局内存的时候，还可以在写出时合并Reduce能力，这部分能力可以在具体的使用场景中参考相关文档来使能。

之前提到了TMA提供了对TMA Descriptor对预取能力，TMA还提供了对数据的全局内存要读取的数据进行预取的能力，使得程序员能够在计算过程中对需要用到的数据进行预取，以提升访存效率。

由于TMA是单线程发起范式，在一个block和warp内有一个线程即可以完成大块数据的拷贝，所以在指令设计方面它属于Uniform指令的范畴，自然地它的参数也为Uniform寄存器，能够减少对general

寄存器的使用。同时为了辅助这种单线程发起拷贝模式，NVidia提供了`elect.sync` PTX指令来更好的实现leader线程的选取。

由于是大块的内存访问，出于对效率的考虑，TMA访问的内存会有对齐的要求，具体的对齐要求如下（参看NV的CUDA编程手册）

Table 23: Alignment requirements for multi-dimensional bulk tensor asynchronous copy operations.

| Address / Size                | Alignment                                                                         |
|-------------------------------|-----------------------------------------------------------------------------------|
| Global memory address         | Must be 16 byte aligned.                                                          |
| Global memory sizes           | Must be greater than or equal to one. Does not have to be a multiple of 16 bytes. |
| Global memory strides         | Must be multiples of 16 bytes.                                                    |
| Shared memory address         | Must be 128 byte aligned.                                                         |
| Shared memory barrier address | Must be 8 byte aligned (this is guaranteed by <code>cuda::barrier</code> ).       |
| Size of transfer              | Must be a multiple of 16 bytes.                                                   |

知乎 @reed

Figure 8. Aligment Requirement for TMA （from NV CUDA Programming Guide）

针对TMA能力，不管是TMA Descriptor的创建、device端对拷贝的发起还是TMA Descriptor的预取、预访问Tensor的预取，以及fence能力等，cute都做了很好的封装，我们后续在完成Hopper上高效GEMM实现时也将使用这些封装。核心封装在cute的/arch/copy\_sm90\_desc.hpp文件和cute/arch/copy\_sm90\_tma.hpp文件中，简单摘录如下，在cute/arch/copy\_sm90\_desc.hpp文件中有对TMA Descriptor的预取的封装

```
void prefetch_tma_descriptor(TmaDescriptor const* desc_ptr);

enum class CacheHintSm90 : uint64_t {
    EVICT_NORMAL = 0x1000000000000000,
    EVICT_FIRST = 0x12F0000000000000,
    EVICT_LAST = 0x14F0000000000000,
};
```

在cute/arch/copy\_sm90\_tma.hpp有对发起copy能力的封装，包含LOAD、SOTRE、REDUCE、1D能力、IM2COL和MULTICAST能力：

```
struct SM90_TMA_LOAD{ _MULTICAST }[_1D/2D/3D/4D/5D]/PREFETCH
struct SM90_TMA_LOAD_IM2COL{ _MULTICAST }[_3D/4D/5D]
struct SM90_TMA_STORE[_1D/2D/3D/4D/5D]
struct SM90_TMA_REDUCE_ADD[_1D/2D/3D/4D/5D]
struct SM90_BULK_COPY_G2S/S2G
```

## 总结

SIMT的编程范式中需要每一个线程参与数据的读写，其需要处理复杂的边界、针对共享内存需要处理由于共享内存bank结构带来而引入的swizzle优化，这回消耗大量的整数计算指令，同时开发效率也会因此而降低，针对这些问题，将上面的这些计算和处理需求封装成独立的硬件单元TMA，其能够高效的完成数据在共享内存和全局内存之间的拷贝，同时可以通过配置方式处理swizzle模式，oob模式，减少了地址的相关计算，提高了编程的效率。这是NVidia对SIMT编程范式的一次突破。如果要使用TMA能力，需要构造TMA Descriptor和发起拷贝两个部分，以全局内存向共享内存拷贝为例详细介绍了相关的指令，并且介绍和解读了开发中可能会经常遇到的可见性问题。cute对该能力有相对全面且完整的封装，利用好cute可以很好的利用其TMA的能力，完成数据的高效拷贝。后续我们将介绍Hopper上另一个重要功能单元WGMMA，最后我们将利用TMA完成高效的数据加载，利用WGMMA完成高效的矩阵运算、利用MBarrier完成高效的协同，借助这三者的力量完成Hopper架构上高效的矩阵计算。

## 参考

[cute 之 Hopper MBarrier](#)

[cute 之 GEMM流水线](#)

[cute 之 Swizzle](#)

[cute 之 Layout](#)

[cute 之 Swizzle](#)

<https://docs.nvidia.com/cuda/cuda-programming-guide/04-special-topics/async-copies.html#using-the-tensor-memory-accelerator-tma>

<https://docs.nvidia.com/cuda/parallel-thread-execution/#data-movement-and-conversion-instructions-bulk-copy>

<https://patents.google.com/patent/US20230289292A1/en>

<https://resources.nvidia.com/en-us-hopper-architecture/nvidia-h100-tensor-c>